

```
# DATA MINNING PROYECT - NAIVE BAYES
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import OrdinalEncoder
from sklearn.naive_bayes import CategoricalNB
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Dataset sizes
dataset_sizes = [22, 100, 1000, 10000, 100000]
base_path = "./Data/dataset"
```

```
#Predicts rows and target
features_columns = ['Director', 'Production', 'User', 'Genre']
target_column = 'Rank'
```

```
# results storage
results = []
```

```
# Function to determine dynamic n_splits
def calculate_n_splits(y, max_splits=10):
    # Ensure that each fold has at least 1 example per class
    min_class_count = y.value_counts().min()
    return min(min_class_count, max_splits)
```

```
for size in dataset_sizes:
    file_path = f"{base_path}{size}.csv"
    print(f"\n=== Evaluating model with dataset of {size} records")
```

```
    # Load dataset
    df = pd.read_csv(file_path)
    X = df[features_columns]
    y = df[target_column]
```

```
    # Categorical coding
    encoder = OrdinalEncoder()
    X_encoded = encoder.fit_transform(X)
```

```
    n_splits = calculate_n_splits(y)
    if n_splits < 2:
        print("Dataset too small for K-Fold. Precision will be used on the entire dataset.")
        model = CategoricalNB(alpha=1.0)
        model.fit(X_encoded, y)
        accuracy = model.score(X_encoded, y)
        cv_scores = np.array([accuracy])
    else:
        cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
        model = CategoricalNB(alpha=1.0)
        cv_scores = cross_val_score(model, X_encoded, y, cv=cv, scoring='accuracy')
```

```
    # Individual report of each fold
```

```

print(f"\nAccuracy results by folds:")
for i, score in enumerate(cv_scores, 1):
    print(f"Fold {i}: {score:.4f}")

mean_acc = cv_scores.mean()
std_acc = cv_scores.std()
print(f"\nMean accuracy: {mean_acc:.4f} | standard deviation: {std_acc:.4f}")

results.append({
    "Dataset_Size": size,
    "Mean_Accuracy": mean_acc,
    "Std_Accuracy": std_acc
})

# individual graphic
plt.figure(figsize=(7,4))
plt.bar(range(1, len(cv_scores)+1), cv_scores, color='#FF7F0E')
plt.axhline(y=mean_acc, color='blue', linestyle='--', label=f'Mean={mean_acc:.3f}')
plt.title(f'Accuracy per fold - Dataset {size}')
plt.xlabel('Fold')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.legend()
plt.show()

```

Global Logarithmic Comparative Chart

```

results_df = pd.DataFrame(results)

plt.figure(figsize=(8,5))
sns.lineplot(
    x="Dataset_Size",
    y="Mean_Accuracy",
    data=results_df,
    marker="o",
    color="#FF7F0E",
    label="Mean accuracy"
)
plt.fill_between(
    results_df["Dataset_Size"],
    results_df["Mean_Accuracy"] - results_df["Std_Accuracy"],
    results_df["Mean_Accuracy"] + results_df["Std_Accuracy"],
    color='#AEC6CF',
    alpha=0.4,
    label="±1 standard deviation"
)

plt.title("Comparison between the size of the dataset and its accuracy")
plt.xlabel("Dataset size (logarithmic scale)")
plt.ylabel("Mean Accuracy")
plt.xscale('log') # Escala logarítmica para eje X
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()

```

```
plt.show()
```